

**1) 주어진 공개키와 유출된 부분 정보를 활용하여  $N$ 을 소인수분해하고, 교과서적 RSA로 암호화된 암호문을 복호화하시오.**

문제에서 주어진 정보를 살펴보면 소수  $p$ 의 일부 비트가 유출되었고 공개키 정보( $N$ ,  $e$ ) 또한 확인할 수 있다. 이러한 경우 Coppersmith Method, Herrmann-May Method 등의 격자 기반 기법 등을 활용하여 나머지 비트를 복원하고, 이를 통해  $N$ 을 소인수분해할 수 있음이 알려져 있다.

해당 방법들은 원래 시프트 다항식과 격자 구조를 수동으로 설계하여 적절한 파라미터를 구하여야 한다. 하지만 'Solving Multivariate Coppersmith Problems with Known Moduli(2025)' 논문과 그 구현체 `cuso`를 활용하면 수치적 파라미터와 최적의 시프트 다항식 선택 전략을 자동화하여 설정할 수 있다. `cuso` 라이브러리를 활용할 때, 풀이자는 다항식, `bound`, 모듈러스만 선언하면 된다.

본 문제에서 유출된 비트 마스크의 구조를 살펴보면 4비트 크기의 홀 2개와 상대적으로 크기가 큰 5개의 홀이 존재한다. 구체적인 오프셋과 길이는 다음과 같다. `cuso` 라이브러리의 실행시간은 다항식에 포함된 변수의 개수에 지수적으로 비례하므로 각각의 홀을 모두 별도의 미지수로 설정하면 합리적인 시간 안에 풀기 어렵다.

```
hole 0: offset=150, len=4
hole 1: offset=265, len=84
hole 2: offset=362, len=58
hole 3: offset=600, len=69
hole 4: offset=682, len=87
hole 5: offset=784, len=46
hole 6: offset=920, len=4
```

그림1. 주어진 MASK의 구조

따라서 홀들을 적당히 병합하여 미지수의 개수를 줄이는 전략을 취한다. 작은 홀 2개는 4비트로 크기가 작지만, 큰 홀들과 비트 위치상 떨어져 있어 함께 병합할 경우 단일 미지수가 처리해야 하는 `bound`가 지나치게 커진다. `bound`는 앞서 살펴본 방법들의 성립 조건과 실행 시간에 주요한 영향을 미치므로, 작은 홀 2개는 최대  $2^8 = 256$  가지의 경우의 수에 대해 브루트포스로 처리하였다. 나머지 5개의 큰 홀은 비트 위치상 인접한 것들끼리 묶어 2개의 미지수  $x_1$ ,  $x_2$ 로 병합하여 표현하였다. 변형한 MASK의 구조와 이에 따른 다항식의 정의는 다음과 같다.

```
hole x1: offset=265, len=155
hole x2: offset=600, len=230
```

그림2. 변형한 MASK의 구조

$$p = p_{masked} + x_1 \times (2^{265}) + x_2 \times (2^{600})$$

각 미지수의 범위로부터 `bound` 크기를 각각 155비트, 230비트로 설정하고  $N = p \cdot q$ 임을 이용하여 모듈러스를 선언하면 `cuso.find_small_roots` 함수를 사용하여  $x_1$ 과  $x_2$ 를 복원할 수 있다.

`cuso.find_small_roots`를 호출할 때, 원본 `cuso`는 설정된 `bound`가 이론적 임계값에 근접하여 첫 격자에서 충분히 짧은 벡터를 얻지 못하거나, 짧은 벡터를 얻더라도 유효한 근으로 이어지지 않는 상황을 대비해, 시프트 다항식을 추가하며 격자 차원을 점진적으로 확장한다.

그러나 본 문제는 변형된 MASK 구조에서 미지수의 총 크기가 355비트로 이론적 한계인 512비트보다 충분히 작으므로 올바른 입력이라면 첫 격자에 사실상 항상 근이 복원된다고 할 수 있다. 역으로 첫 격자에서 유효한 근을 찾지 못하면 그 입력은 잘못된 입력이라 판단할 수 있다. 잘못된 입력에서는 어떤 크기의 격자라도 근이 존재하지 않으므로, 격자를 확장하더라도 유효한 근을 얻을 수 없어 연산 낭비가 발생한다.

이에 첫 격자축소 이후 근을 찾지 못하면 격자를 확장하지 않고 해당 후보를 포기하도록 cuso를 수정하여, 잘못된 브루트포스 후보에 연산을 낭비하지 않게 하였다. 해당 수정은 'cuso.patch'로 적용하였다. 이 설계에서 (155,230) bound의 한 후보를 푸는 데 대부분 500초 이내가 소요되므로 최악의 경우에도 약 36시간 걸린다. 이는 충분히 현실적인 시간 안에 해결할 수 있다.

풀이 결과 후보 256개중 186번째 후보에서 다음과 같이 비밀키 p, q를 찾고 이를 바탕으로 복호화를 진행해 flag를 획득 할 수 있다.

p=

```
0xffa360d46885c534d186538179633fafc2c0548a2e24a2c1c878569f522939e38ff75ead1bcd442a834974a52e3ac66c1
bc2ee00d63e2de4c436d2ca740a624699e1a1af94045c63261323cb723a7ba1b3c00bbbb6a4e534c11469e73ddb2e5
0b0bc2461fcbac0726360c2c0ac9450a9a892cbf1d98ceee48827591ccc593c9
```

q=

```
0xe557fa8670389cb60c84416a65742a74fd11ed33b1631f787e92b90887b5391dacba00a386911bf8a8fbd57430b9b2
6e455329405ffe289e20616fe3b5562ea9b533f8f8db94bb8dcd280a6af056108e176008d3655428ad0ac6396318ba0f6
efe496eac3f8585675bfed67081e0c518be5685e4daf7060abe1c58b73cc5f1e9
```

```
plain = FLAG{d1rty_b1t_l34k_c0pp3rsm1th_m33ts_str4t3gy}
```

cuso 라이브러리 패치 내용은 'cuso.patch', p를 복호화 하는데 사용한 코드는 'solve.py', 복호화 코드는 'decrypt.py'에서 확인할 수 있다.